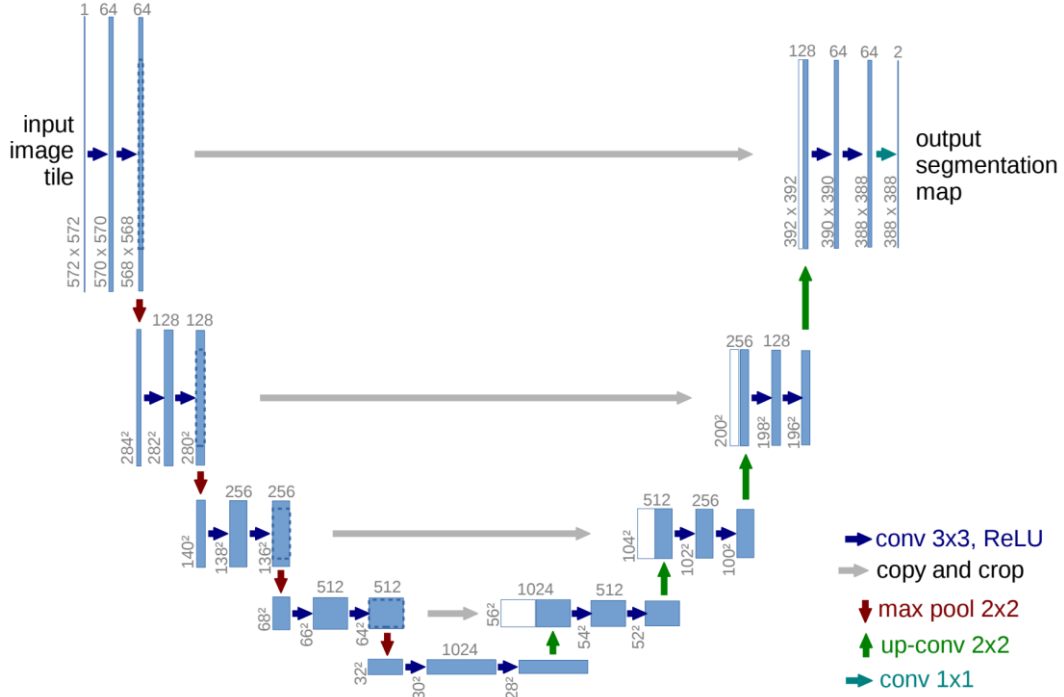


U-Net

导引：



假设输入 $x \in \mathbb{R}^{B \times 3 \times 256 \times 256}$

第一层编码 $E_1 = \text{ConvBlock}(x)$

得到 $E_1: 64 \times 256 \times 256$

下采样: $D_1 = \text{Down}(E_1)$

得到: $64 \times 128 \times 128$

第二层: $E_2 = \text{ConvBlock}(D_1)$

得到: $128 \times 128 \times 128$

继续: $E_3: 256 \times 64 \times 64$; $E_4: 512 \times 32 \times 32$

瓶颈: $M: 1024 \times 16 \times 16$

解码: $U_4 = \text{Up}(M)$

得到: $512 \times 32 \times 32$ 与 E_4 连接 $F_4 = [U_4; E_4]$

得到: $1024 \times 32 \times 32$

卷积: $Y_4: 512 \times 32 \times 32$

继续: $Y_3: 256 \times 64 \times 64$; $Y_2: 128 \times 128 \times 128$; $Y_1: 64 \times 256 \times 256$

最后使用 1×1 卷积: $Y = \text{Conv}_{1 \times 1}(Y_1)$

得到输出: $Y: C_{\text{out}} \times 256 \times 256$

(<https://arxiv.org/pdf/1505.04597>)

上一节我们已经得到 DDPM 的训练目标:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2]$$

其中：

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$$

现在出现了一个问题：

用什么神经网络实现 $\epsilon_\theta(x_t, t)$ 来预测噪声？经典 Diffusion 通常使用的就是 U-Net，因此，在扩散模型中，U-Net 的任务不是直接生成图像，而是接收当前的噪声图像 x_t 和时间步 t ，输出噪声预 $\hat{\epsilon} = \epsilon_\theta(x_t, t)$

整体过程为： $(x_t, t) \rightarrow \text{U-Net} \rightarrow \hat{\epsilon}$

$$\text{然后使用预测噪声完成一步去噪 } x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \hat{\epsilon} \right) + \sigma_t z$$

U-Net 最初用于医学图像分割。例如输入一张细胞图像： $x \in \mathbb{R}^{H \times W \times 3}$

网络需要判断每一个像素属于背景；细胞；病变区域；其他类别。输出： $y \in \mathbb{R}^{H \times W \times C}$

其中 C 表示类别数量。

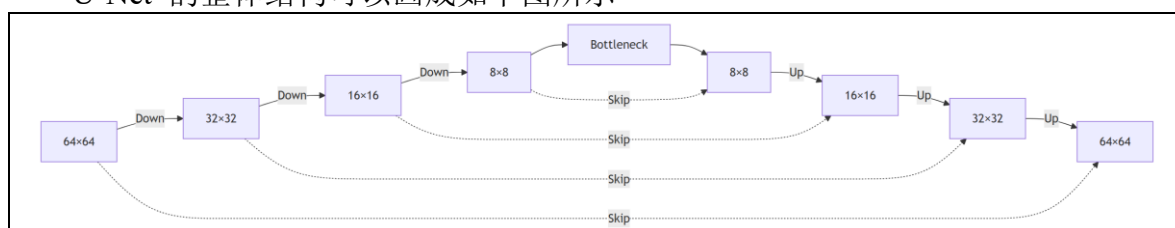
普通图像分类只输出一个类别： $\text{CNN}(x) \rightarrow \text{“猫”}$

而图像分割需要对每个像素分类： $\text{U-Net}(x) \rightarrow \text{每个像素的类别}$

因此输出图像需要保持和输入相同的空间尺寸。

例如 $512 \times 512 \times 3$ 经过 U-Net 后 $512 \times 512 \times 2$ ，空间大小仍然是 512×512

U-Net 的整体结构可以画成如下图所示



左边不断下采样。右边不断上采样。中间通过跳跃连接连接。整体形状类似英文字母U，所以叫U-Net

U-Net 可以分为 Encoder + Bottleneck + Decoder，还包含一个非常重要的结构 Skip Connection，即跳跃连接。

完整表示为 $x \rightarrow \text{Encoder} \rightarrow \text{Bottleneck} \rightarrow \text{Decoder} \rightarrow y$

编码器负责降低空间分辨率，提取高级语义；

解码器负责恢复空间分辨率，生成像素级结果；

跳跃连接负责将编码器中的细节信息传递给解码器。

假设输入： $x \in \mathbb{R}^{B \times 3 \times 256 \times 256}$ ，其中 B 是 Batch Size；输入通道 $C = 3$ ；高度 $H = 256$ ；宽度 $W = 256$ 。

编码器不断执行卷积-激活-卷积-激活-下采样

例如： $256 \times 256 \rightarrow 128 \times 128 \rightarrow 64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16$

空间尺寸不断减小。

通道数量不断增加 $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ ，因此特征变化可能是 $3 \times 256 \times 256$

经过第一层后 $64 \times 256 \times 256$ ；下采样 $64 \times 128 \times 128$
 继续卷积 $128 \times 128 \times 128$ ，继续下采样 $128 \times 64 \times 64$ ；
 最终 $512 \times 16 \times 16$

假设输入特征 $X \in \mathbb{R}^{C_{in} \times H \times W}$ ；卷积核 $W \in \mathbb{R}^{C_{out} \times C_{in} \times K \times K}$ ；输出 $Y \in \mathbb{R}^{C_{out} \times H' \times W'}$

第 c_o 个输出通道中位置 (i, j) 的结果为 $Y_{c_o, i, j} = b_{c_o} + \sum_{c_i=1}^{C_{in}} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} W_{c_o, c_i, u, v} X_{c_i, i+u, j+v}$

其中 c_i 表示输入通道； c_o 表示输出通道； u, v 表示卷积核位置。

[例如使用 3×3]

卷积输出尺寸 $H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1$ ，其中 K 是卷积核大小； P 是 Padding。

S 是步长 Stride。

假设 $H_{in} = 64$ ，使用 $K = 3$ ；Padding 为 $P = 1$ ；Stride 为 $S = 1$

得到 $H_{out} = \frac{64 + 2 - 3}{1} + 1$ ；因此 $H_{out} = 64$ ；所以 3×3 卷积配合 $P = 1$ 可以保持图像尺寸不变。

现代 Diffusion U-Net 通常大量使用 $3 \times 3, S = 1, P = 1$

上采样如何实现？

方法一：转置卷积

$X_{up} = ConvTranspose(X)$

例如 $16 \times 16 \rightarrow 32 \times 32$ ，转置卷积拥有可学习参数。

转置卷积不是卷积的逆运算。它只是通过特殊的卷积结构增加空间分辨率。

方法二：插值加卷积

先插值 $X' = Upsample(X)$

例如 $16 \times 16 \rightarrow 32 \times 32$ ，然后卷积 $X_{up} = Conv(X')$

即 $X_{up} = Conv(eUpsample(X))$

Diffusion U-Net 中经常使用 Nearest Upsampling + 3×3 Conv，这样可以减少转置卷积可能出现的棋盘格伪影。

假设编码器执行 $256 \times 256 \rightarrow 128 \times 128 \rightarrow 64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16$ ，大量空间信息可能在下采样过程中丢失。

如果仅依赖 16×16 的特征恢复 256×256 输出可能比较模糊。

因此 U-Net 引入 Skip Connection

假设编码器某层输出 $E_l \in \mathbb{R}^{B \times C_e \times H \times W}$ ；解码器恢复到相同尺寸 $D_l \in \mathbb{R}^{B \times C_d \times H \times W}$ ；将两者连接 $F_l = Concat(E_l, D_l)$ ，连接发生在通道维度。

因此 $F_l \in \mathbb{R}^{B \times (C_e + C_d) \times H \times W}$

U-Net 和 ResNet 的跳跃连接有什么区别？

这两个都叫 Skip Connection，但是操作不同。

U-Net 常使用 $Y = \text{Concat}(X, F(X))$ 沿通道维度拼接。

ResNet 使用 $Y = X + F(X)$ 逐元素相加。

[经典 U-Net 通过图像获得像素分类结果，例如 $x \in \mathbb{R}^{3 \times 256 \times 256}$ 输出 $y \in \mathbb{R}^{2 \times 256 \times 256}$ ，Diffusion U-Net 利用 $(x_t, t) \rightarrow \hat{\epsilon}$ 输入噪声图 $x_t \in \mathbb{R}^{3 \times 256 \times 256}$ ；输出预测噪声 $\hat{\epsilon} \in \mathbb{R}^{3 \times 256 \times 256}$ ，输入和输出形状完全相同 $\text{Shape}(x_t) = \text{Shape}(\hat{\epsilon})$

假设同一张图像处于不同时间 x_{10} 只有少量噪声。而 x_{900} 几乎完全是随机噪声。网络的去噪策略应该不同。对于 x_{10} 只需要去除少量噪声，保留大部分图像细节。对于 x_{900} 需要从全局结构开始恢复。因此网络必须知道当前处于第几个扩散时间步骤。

输入变成 $\epsilon_\theta(x_t, t)$ ，而不是 $\epsilon_\theta(x_t)$

时间 t 只是一个整数。例如 $t = 500$ ，不能简单地将数字 500 直接输入卷积网络。通常先使用正弦位置编码。

$$\text{定义 } PE(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right); PE(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

其中 d 是时间嵌入维度。例如 $d = 128$ 则 $t \rightarrow e_t \in \mathbb{R}^{128}$ 不同维度使用不同频率；低频维度变化缓慢；高频维度变化快速；这样网络可以区分不同时间步。

时间首先经过多层感知机 $e_t = \text{MLP}(PE(t))$ ，例如： $e_t \in \mathbb{R}^{512}$

某一层图像特征为 $H \in \mathbb{R}^{B \times C \times H \times W}$

先将时间特征投影 $e'_t = W_t e_t + b_t$ ，得到 $e'_t \in \mathbb{R}^C$ ，扩展空间维度 $e'_t \in \mathbb{R}^{B \times C \times 1 \times 1}$

然后广播相加得到 $H' = H + e'_t$

虽然 e'_t 只有 1×1 ，但是广播以后会加到所有空间位置 $H'_{b,c,i,j} = H_{b,c,i,j} + e_{t,b,c}$ ，因此每个卷积模块都知道当前时间。]

综上，经典 U-Net 为 $Y = \text{Decoder}(\text{Encoder}(X))$

带跳跃连接 $D_l = \text{Conv}([\text{Up}(D_{l+1}); E_l])$

其中 E_l 是编码器特征。

时间嵌入 $e_t = \text{MLP}(\text{SinusoidalEmbedding}(t))$

Diffusion U-Net 为 $\hat{\epsilon} = \epsilon_\theta(x_t, t)$

$$\text{训练目标为 } \mathcal{L} = \mathbb{E} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, t \right) \right\|_2^2 \right]$$

$$\text{反向去噪 } x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$